

SafeHer AI — Women's Safety Companion

Software Requirements Specification (SRS)

TASK 1: TEAM SELECTION

#	Name	Student ID
1	Hafsa Khurram	BSCS24035
2	Rania Junaid	BSCS24127

TASK 2: SOFTWARE REQUIREMENTS SPECIFICATION

1. Preface

This document is the Software Requirements Specification (SRS) for **SafeHer AI**, a mobile app built to help women feel safer, especially when they are out alone. It follows the structure from Ian Sommerville's *Software Engineering*, 10th edition, Figure 4.17.

This document is written for the development team, course instructors, and evaluators. It explains what the app needs to do and the rules it needs to follow.

A key principle running through every part of this design is that **the app must not be weaponisable**. Recordings are never handed over to the user directly. Instead, an AI model reviews each emergency after it ends and decides whether the situation was genuinely unsafe. Only then can a recording ever be submitted, and even then only to a designated authority contact rather than shared freely. This protects innocent people from being falsely implicated. Requirements marked **[ML]** involve a machine learning model.

Version History:

Version	Date	Description
1.0	2026	First version of the SRS document

2. Introduction

a) Purpose

The purpose of this document is to clearly lay out what SafeHer AI is supposed to do and how it should behave. SafeHer AI is a mobile app that helps women stay safe in situations where they feel threatened or uncomfortable, especially when asking for help openly could make things worse.

The problem we are solving is straightforward. When a woman feels unsafe, she often cannot take out her phone and call someone without drawing attention to herself. Most existing safety apps require you to visibly tap through screens, which can be dangerous in the wrong situation. SafeHer AI works differently. It lets the user trigger help silently, without touching the screen at all, using either a button shortcut or a spoken phrase. Once triggered, the app records audio and video, shares the user's live location with trusted contacts, and can even fake an incoming phone call to help the user leave an uncomfortable situation naturally.

The app also uses AI to learn which areas of a city tend to be less safe at night and quietly warns the user before they walk into one.

Importantly, this app is designed so that it cannot be turned against innocent people. Recordings stay locked inside the app at all times. After an emergency ends, a separate AI model reviews the recording and the context around it, for example how long the emergency lasted, what was heard in the audio, and whether the location was already flagged as risky, and decides whether the situation was genuinely unsafe. A recording can only ever be submitted to an authority if that review confirms it. **The user cannot export or share recordings on their own.**

This document serves as the main reference for building, testing, and evaluating the application.

b) Scope

SafeHer AI is an Android app, with plans to support iOS later, that runs quietly in the background on the user's phone. It covers six main areas:

Silent Emergency Activation: The user can start an emergency without unlocking their phone, either by pressing the power button twice quickly or by saying a secret phrase they can set themselves.

Recording Evidence: As soon as an emergency is triggered, the app silently starts recording audio and video and saves the files securely in a private area of the device that only the app can access. The user cannot open or share these files directly. Screen recording and screenshotting will be prohibited.

Sharing Live Location: The app sends the user's real time location to their chosen emergency contacts and keeps updating it until the user signals they are safe.

Fake Call Feature: The user can trigger a fake incoming phone call, complete with a ringing tone, and a custom caller name, so they have a natural reason to walk away from a situation without it looking suspicious.

AI Risk Zone Alerts [ML]: A machine learning model looks at crime patterns in the city and alerts the user when they enter a higher risk area during nighttime hours.

Situation Classification [ML]: After every emergency ends, a **separate** AI model reviews the audio, the duration, how the emergency was deactivated, and the location context to decide whether the situation was genuinely unsafe. This classification controls whether a recording can ever be submitted to authorities. **It is the core feature that stops the app from being misused against innocent people.**

To be clear about what the app will not do: it will not replace emergency services. It will not identify or record bystanders. It will not give the user free access to recordings. It will not allow any recording to leave the app without an AI confirmed unsafe classification first.

3. Glossary

Term	Definition
SafeHer AI	The name of the app described in this document.
Distress Activation	The moment the user triggers the emergency, either through the button shortcut or the voice trigger.

Emergency Contact	A trusted person the user has added to the app who will receive alerts and the user's live location during an emergency.
Trigger Phrase	A secret word or phrase the user sets up. Saying it out loud starts the emergency without any visible action on screen.
Risk Zone	An area that the AI model has flagged as higher risk based on past crime data and the time of day.
Fake Call	A simulated phone call the app creates to give the user a believable excuse to leave a situation.
Situation Classification	The AI process that runs after every emergency ends. It looks at multiple signals and decides whether the situation was genuinely unsafe.
Unsafe Classification	The result when the model determines a real threat was likely present. Only incidents with this result can be submitted to an authority.
Inconclusive or Safe Classification	The result when the model cannot confirm a genuine threat. The recording is held internally and deleted automatically after 72 hours.

n	
Authority Contact	A pre registered contact, such as a campus security office or trusted institution, that can receive a classified incident submission. This is different from a regular emergency contact.
ML Model	A machine learning model, a type of AI trained on data to recognise patterns and make decisions.
Background Service	A part of the app that keeps running on the phone even when the user is not actively using it.
Geofencing	Drawing a virtual boundary on a map so the app can detect when the user enters or leaves that area.
GPS	The technology the phone uses to find the user's exact location.
Nighttime Hours	For this app, this means between 8:00 PM and 6:00 AM local time.
SRS	Software Requirements Specification, which is this document.

4. User Requirements Definition

These requirements describe what the app needs to do from the user's point of view, written in plain language that anyone can understand.

a) Functional Requirements

UR-F1: Emergency Activation via Button The user should be able to start an emergency by quickly pressing the power button twice, even when the phone is locked and the app is not open on screen.

UR-F2: Emergency Activation via Voice The user should be able to set a personal secret phrase. When they say it out loud, the app should start the emergency with no visible action required on the phone.

UR-F3: Silent Recording Once an emergency starts, the app should quietly begin recording audio and video. The user does not need to do anything else. These recordings are stored privately inside the app and the user cannot open, copy, or share them directly.

UR-F4: Live Location Sharing The app should automatically send the user's location to their emergency contacts as soon as an emergency is triggered, and keep sending updates until the user confirms they are safe.

UR-F5: Fake Incoming Call The user should be able to make the phone look like it is receiving a real call, with a ringtone, vibration, and a contact name on screen, so they have a natural excuse to leave a situation without it looking suspicious. The fake call should not appear in the

phone's call history.

UR-F6: Risk Zone Alerts When the user is out at night and enters an area the AI has flagged as higher risk, the app should give them a quiet alert. The user should also be able to see a map showing risky areas near them.

UR-F7: Situation Classification After Emergency After every emergency ends, the app should automatically analyse what happened using the audio, the duration of the emergency, how it ended, and the location. It should then tell the user in simple language whether the situation was classified as genuinely unsafe, inconclusive, or safe.

UR-F8: Submitting to Authorities If the AI classifies a situation as genuinely unsafe, the user should have the option to submit the recording and incident report to a pre registered authority contact. If the classification comes back as inconclusive or safe, this option should not be available at all.

UR-F9: Managing Emergency Contacts The user should be able to add, edit, and remove the people they want notified in an emergency. Each contact should be reachable by SMS and through the app.

UR-F10: Incident Log The app should keep a secure internal record of every emergency, including the classification result. The user can view past incidents but cannot export the raw recordings.

b) Non-Functional Requirements

UR-NF1: Discretion Everything the app does in an emergency should be invisible to anyone nearby. Nothing should appear on screen that a bystander could notice.

UR-NF2: Reliability The emergency trigger should work correctly at least 99 times out of 100, as long as the phone is on and the app is running in the background.

UR-NF3: Privacy and Misuse Prevention Recordings must never be accessible to the user as free files. They stay locked inside the app. The only way a recording can ever go anywhere outside the app is through an AI confirmed unsafe classification followed by a deliberate authority submission. **The app must not be something that can be pointed at an innocent person and used against them.**

UR-NF4: Transparency of Classification The user should always be able to see what the AI decided about their emergency and why, explained in simple language. The result should never just be a label with no explanation.

UR-NF5: Easy to Use A new user should be able to finish setting up the app, choose a trigger phrase, and add at least one emergency contact in under five minutes, with no technical knowledge needed.

UR-NF6: Battery Friendly The app running in the background should not drain more than 3% of the phone's battery per hour, so the phone is still usable when the user actually needs it.

5. System Requirements Definition

These requirements are more detailed and technical. They are written for the developers who will actually build the system.

a) Functional Requirements

SR-F1: Power Button Listener The app should run a background process that watches for the power button being pressed twice within 500 milliseconds. When that is detected, it immediately kicks off the emergency sequence. This needs to work even when the phone screen is off.

SR-F2: Voice Trigger Detection [ML] The app should run a small, efficient speech recognition model on the phone at all times. When it hears the user's saved trigger phrase with at least 90% confidence, it fires the emergency. False alarms should happen no more than once every eight hours of background listening. All of this runs on the phone itself and no audio is ever sent to an outside server.

SR-F3: Silent Recording Module The moment an emergency starts, the app turns off any camera shutter sounds or flash and begins recording video at 720p and audio at standard quality. Recordings are saved in 60 second encrypted chunks using AES-256 in a private app folder that cannot be accessed through the phone's file manager or any other app. The user has no direct access to these files at any point.

SR-F4: Location Broadcasting Service The app gets a GPS reading at least once every 15 seconds. When an emergency starts, it sends the user's coordinates to all emergency contacts through the internet first and by SMS if there is no connection. It keeps sending until the user deactivates using their PIN or fingerprint.

SR-F5: Fake Call Engine The app generates a full screen incoming call screen showing a user defined name, plays a ringtone, and vibrates. The user can answer it and end it when ready. Nothing from this call is saved to the phone's native call log.

SR-F6: Risk Zone Classification Model [ML] A pre trained geospatial model takes the user's current location, the time of day and the day of the week as inputs and returns a risk score between 0.0 and 1.0. Any area scoring 0.65 or above is treated as a risk zone. The model updates itself over Wi-Fi no more than once per day.

SR-F7: Geofence Alert Dispatcher When the user enters a risk zone between 8 PM and 6 AM, the phone vibrates three short times. A quiet notification appears that does not show any sensitive text on the lock screen.

SR-F8: Situation Classification Model [ML] After every emergency ends, the app runs a classification model on the completed incident. The model looks at four things: the audio from the recording, checking for raised voices, distress sounds, or threatening speech; the duration, since very short emergencies under 30 seconds are weighted toward accidental; how the emergency ended, since a calm PIN deactivation suggests the user regained safety normally while the phone going offline or losing power suggests a more serious situation; and the

location, checking whether the area was already flagged as a risk zone during nighttime hours. Based on all four signals, the model produces one of three results: Unsafe, Inconclusive, or Safe. The result and a plain language explanation are saved with the incident record and shown to the user.

SR-F9: Controlled Authority Submission Only when the situation classification returns Unsafe does the app unlock a submission option. The user can then choose to send the encrypted incident report and recording to a pre registered authority contact. The user never receives the recording file themselves at any point in this process. The submission is one way and permanently logged.

SR-F10: Contact Notification Gateway Emergency contacts are notified through Firebase Cloud Messaging if they have the app installed, and by SMS if they do not. Every message includes the user's name, a Google Maps location link, and a distress flag.

SR-F11: Encrypted Incident Log The app keeps a local database encrypted with SQLCipher. Each record stores the timestamp, activation method, GPS coordinates, duration, how the emergency ended, the situation classification result and confidence score, and whether an authority submission was made. The user can read these records inside the app but cannot export the raw data or recordings.

SR-F12: Onboarding and Configuration A guided setup flow collects the user's name, between one and five emergency contacts, a trigger phrase checked for minimum complexity, an optional authority contact, and a PIN or fingerprint lock. All changes to settings require the user to confirm their identity again first.

b) Non-Functional Requirements

SR-NF1: Activation Speed Recording must start within 2 seconds of a valid emergency trigger under normal phone operating conditions.

SR-NF2: Everything Runs on the Device All three ML models, the voice trigger, the risk zone classifier, and the situation classifier, must run entirely on the phone. No audio, video, or location data is sent to any external server for any of these models to work.

SR-NF3: Recording Access Control Recorded media must be stored in a private folder that the user, other apps, and the phone's file system cannot access. The only way these files can ever go anywhere is through the controlled authority submission process in SR-F9, and only after an Unsafe classification.

SR-NF4: Data Security All sensitive data stored on the phone must be encrypted with AES-256. All data sent over the internet must use a secure connection, specifically TLS 1.3 or higher. The app must only ask for the permissions it actually needs.

SR-NF5: Device Compatibility The app must work on Android 9.0 and above. Any limitations caused by Android 12+ battery restrictions must be clearly explained to the user rather than silently failing.

SR-NF6: Performance and Battery All background processes together must not use more than 4% of the phone's processing power on average and must not drain more than 3% of the battery per hour.

SR-NF7: Works Without Internet The core features, which are triggering the emergency, recording, and sending location by SMS, must all work without a data connection. Features that need the internet must fail quietly and let the user know what is not working.

SR-NF8: Keeping the Models Up to Date All three ML models must be set up so they can be updated on their own without needing a full app update. Any updated model must pass a minimum accuracy check of 88% on test data before it goes out to users.

SR-NF9: Classification Transparency The classification result shown to the user must include a plain language explanation of which signals contributed to the outcome. For example, it might say something like: "The emergency lasted under 20 seconds and ended with a calm PIN deactivation, which suggests this was likely accidental."

SR-NF10: Auto Deletion All recordings and incident data are automatically deleted after **72 hours** unless the classification returned Unsafe and the user is actively considering an authority submission. The user cannot override this default for Inconclusive or Safe results.

6. System Models: Detailed Use Cases

Detailed use case descriptions are provided below.

Use Case UC-01: Activate Emergency Mode via Hardware Shortcut

Field	Details
Use Case ID	UC-01
Use Case Name	Activate Emergency Mode via Hardware Shortcut
Actor(s)	Primary User

Description	The user starts an emergency by pressing the power button twice quickly. The app begins recording and shares their location without the phone needing to be unlocked or the app needing to be opened.
Preconditions	The app's background service is running. At least one emergency contact has been added. The phone has at least 50 MB of free storage.

Main Flow	1. The user feels unsafe and presses the power button twice within 500 ms. 2. The app detects this and starts the emergency process. 3. The app silently suppresses camera sounds and starts recording audio and video into its private encrypted storage. 4. The app gets the user's current GPS location. 5. The app sends the user's name, a distress flag and a location link to all emergency contacts via the app and by SMS if needed. 6. The app saves a record of the activation including the time, method, and location in its encrypted log. 7. The app keeps recording and sending location updates every 15 seconds.
Alternate Flow	3a. If the app does not have camera permission, it records audio only and notes this in the log. 5a. If there is no internet connection, the app sends the location by SMS instead.
Exception Flow	2a. If the background service has been shut down by the phone's OS, the shortcut will not work. The app displays a persistent reminder to keep the service active. 4a. If GPS is not available, the last known location is sent and marked as approximate in the message to contacts.
Postconditions	Emergency mode is on. Silent recording has started in private storage. All emergency contacts have received an alert with a live location link.

Use Case UC-02: Activate Emergency Mode via Voice Trigger

Field	Details
Use Case ID	UC-02

Use Case Name	Activate Emergency Mode via Voice Trigger
Actor(s)	Primary User
Description	The user says their secret trigger phrase out loud. The on device AI model hears it and starts the emergency with nothing visible happening on the phone screen.
Preconditions	Background audio monitoring is active. The user has set a trigger phrase. The app has microphone permission.

Main Flow	1. The voice model runs quietly in the background, listening at low power. 2. The user says their trigger phrase. 3. The model recognises it with at least 85% confidence and fires the emergency signal. 4. The app follows the same steps as UC-01, steps 3 to 7. 5. The activation is saved in the log as triggered by voice.
Alternate Flow	3a. If confidence is between 70% and 84%, the phone vibrates once and waits up to 5 seconds for a second utterance before activating. 3b. If confidence is below 70%, nothing happens and the model keeps listening.
Exception Flow	1a. If the audio service has been paused by the phone's battery saver, the app tries to restart it and sends the user a quiet notification asking them to whitelist the app in battery settings.
Postconditions	Emergency mode is on through the voice triggered route. Nothing visible changed on the phone screen during activation.

Use Case UC-03: Trigger Fake Incoming Call

Field	Details
Use Case ID	UC-03
Use Case Name	Trigger Fake Incoming Call
Actor(s)	Primary User
Description	The user makes the phone look like it is receiving a real call so they have a believable, natural reason to leave an uncomfortable situation.

Preconditions	The app is installed and set up. A fake caller name has been chosen, with the default being Mom. The phone is not on silent, or audio override is turned on in settings.
----------------------	--

Main Flow	1. The user taps the Fake Call button in the app or on a home screen widget. 2. The phone shows a realistic full screen incoming call screen with the saved caller name. 3. The phone rings and vibrates. 4. The user answers the call and the screen switches to an active call view. 5. Optionally, the app plays soft background conversation audio through the earpiece. 6. The user ends the call and the screen closes. 7. Nothing is saved to the phone's call history.
Alternate Flow	1a. The user can schedule the fake call to trigger automatically after a chosen delay of between 1 and 10 minutes, which is useful when they know in advance they may need an exit.
Exception Flow	2a. If a real call comes in while the fake call is showing, the fake screen closes immediately and the real call is shown instead.
Postconditions	The fake call has ended. No record of it exists in the call log. The user is back on the app's main screen.

Use Case UC-04: Receive Risk Zone Alert

Field	Details
Use Case ID	UC-04
Use Case Name	Receive Risk Zone Alert
Actor(s)	Primary User; Risk Zone ML Model
Description	The user is out at night and walks into an area the AI has flagged as higher risk. The phone vibrates quietly to warn them with nothing showing on the lock screen.

Preconditions	Location tracking is running in the background. The local time is between 8:00 PM and 6:00 AM. The risk model was updated within the last 24 hours. Risk alerts are turned on in settings.
----------------------	--

Main Flow	1. The app receives a GPS update. 2. It passes the location, time of day, and day of week to the risk model. 3. The model returns a score of 0.65 or above. 4. The phone vibrates three short times. 5. A quiet notification appears saying something like: "You have entered an area with elevated safety activity nearby. Stay aware." This does not appear on the lock screen. 6. If the user opens the notification, they see a map with the risk area highlighted and the option to share their location with a contact.
Alternate Flow	6a. The user dismisses the notification. The app logs the alert and will not send the same alert again for at least 10 minutes. 6b. The user taps Share Location Now. The app sends their live location to a chosen contact without activating full emergency mode.
Exception Flow	3a. If the model cannot run, the app falls back to a saved list of known risk coordinates and alerts based on proximity. 2a. If GPS accuracy is below 50 metres, the app waits for a better reading before running the model.
Postconditions	The user is aware they have entered a risk zone. The alert is saved in the incident log.

Use Case UC-05: Deactivate Emergency Mode

Field	Details
Use Case ID	UC-05
Use Case Name	Deactivate Emergency Mode
Actor(s)	Primary User
Description	Once the user is safe, they turn off emergency mode. The app stops recording, stops sharing their location, tells their contacts they are okay, and then runs the situation classification on what just happened.

Preconditions	Emergency mode is currently active. The user has their phone and can open the app.
----------------------	--

Main Flow	1. The user opens SafeHer AI. 2. The screen shows an active emergency view with a clear I Am Safe button. 3. The user taps I Am Safe. 4. The app asks for PIN or fingerprint confirmation to make sure it is really the user. 5. Once confirmed, the app stops all recording and saves the final media files in private storage. 6. The app sends a User is Safe message to all emergency contacts. 7. The app stops sending location updates. 8. The app immediately begins running the situation classification on the completed emergency. 9. The app returns to the normal idle screen with a small notification saying classification is in progress.
Alternate Flow	4a. If the PIN or fingerprint fails twice in a row, emergency mode stays on and an extra alert goes to contacts flagging that a failed deactivation attempt was made.
Exception Flow	1a. If the user cannot access the app because their phone was taken or damaged, emergency mode stays active until the phone powers off. The abnormal ending is recorded and weighted by the classification model in UC-07.
Postconditions	Emergency mode is off. Contacts have been told the user is safe. The situation classification process has started on the incident.

Use Case UC-06: Situation Classification After Emergency

Field	Details
Use Case ID	UC-06
Use Case Name	Situation Classification After Emergency
Actor(s)	System (Situation Classification ML Model); Primary User

Description	After every emergency ends, the app automatically analyses the incident using four signals and produces a classification of whether the situation was genuinely unsafe. This is the feature that stops the app from being used to falsely implicate innocent people, because recordings can never leave the app without this review confirming a real threat was present.
--------------------	---

Preconditions	An emergency has just ended through UC-05. The recording and incident data are in private encrypted storage.
Main Flow	1. The app passes the completed incident to the situation classification model. 2. The model analyses four signals. First it checks the audio for raised voices, distress sounds, threatening speech, or unusual silence. Second it looks at the duration, since emergencies under 30 seconds are weighted toward accidental. Third it looks at how the emergency ended, since a calm PIN deactivation suggests the user got to safety normally while the phone going offline or losing battery suggests something more serious happened. Fourth it checks whether the location was already a flagged risk zone during nighttime hours. 3. Based on all four signals, the model produces one of three results: Unsafe, Inconclusive, or Safe, along with a confidence score. 4. The result and a plain language explanation are saved to the incident log. 5. The user is shown the result in simple language that explains which signals led to it.
Alternate Flow	5a. If the user disagrees with an Inconclusive result, they can flag it for review. This does not unlock the recording but sends a notification to their pre registered authority contact if one exists.
Exception Flow	1a. If the model fails to run due to a technical issue, the incident is logged as Unclassified. The recording is kept for an extra 48 hours beyond the usual 72 hour window to give the model time to retry.
Postconditions	The incident has a classification label and plain language explanation saved in the log. If the result is Unsafe, the authority submission option is unlocked for the user. If the result is Inconclusive or Safe, no submission option is shown and the recording will auto delete after 72 hours.

TASK 3: REQUIREMENTS ELICITATION AND ANALYSIS

3.1 Elicitation Technique

To figure out what SafeHer AI should do and how it should work, we used two methods: **Brainstorming** and **Analysis of Existing Systems**. We chose both together because this project has two sides to it, a human problem that needed creative thinking and a technical one that needed research.

Brainstorming

We ran brainstorming sessions around one simple question: *"If a woman feels unsafe right now, what does she actually need in the next 30 seconds?"* Keeping it that immediate and personal helped us stay grounded in real situations instead of abstract features.

This is where most of the core ideas came from including the voice trigger, the fake call, and the silent recording. But the most important thing brainstorming surfaced was a problem we had not originally planned for: what stops someone from pointing this app at an innocent person and using it against them? If a user can trigger a recording anywhere and then share it freely, the app becomes a tool for false accusations just as easily as a safety tool. Spotting that risk early let us design around it from the beginning rather than trying to patch it later.

The answer we arrived at was a situation classification model that reviews every incident before any recording can ever leave the app. This also addressed the privacy and monitoring concerns raised by our professor, because the app is not passively watching anyone. Recording only happens when the user triggers it, and even then the recording stays locked unless the AI review confirms a real threat was there.

Analysis of Existing Systems

We looked at three categories of existing tools:

Dedicated safety apps like bSafe and Noonlight showed us that live location sharing and emergency alerts are features users genuinely rely on. But they also showed consistent gaps. Every single one requires visible interaction to trigger. None have a voice activation option. None do any kind of post incident review. And crucially, they all hand over recordings or data directly to the user with no safeguards. That last gap became the main motivation behind SR-F8 and UC-06.

Built in phone safety features like Android Emergency SOS confirmed that a hardware button shortcut is both technically possible and already familiar to users, which gave us confidence in SR-F1.

Research into audio based distress detection and geospatial crime prediction confirmed that both the risk zone model and the situation classification model are realistic to build at a student project scale using open datasets and existing pre trained model architectures. Audio classification specifically, detecting raised voices and distress cues, is a well established area in machine learning with accessible tools available, which made SR-F8 a realistic goal rather than an overreach.

3.2 Stakeholder Identification

Primary Stakeholder: Women Users (Ages 16 to 45)

The main users are women who travel on their own, to university, to work, at night, or through unfamiliar areas. They do not need any technical background. The app has to work for anyone with a basic smartphone.

Their key problems that the ML features directly address:

Not being able to call for help without being seen doing it. The voice trigger model (SR-F2)

solves this entirely. A simple rule based approach could not do this reliably at low power with an acceptable rate of false alarms. A trained ML model is what makes it actually workable.

Not knowing which areas to avoid at night. The risk zone model (SR-F6) turns publicly available crime data into a real time alert the user does not have to think about or research themselves.

Fear that a report will not be taken seriously without evidence. The situation classification model (SR-F8) helps with this from both sides. It supports genuine incidents with a structured AI review, and it keeps the overall system credible by making sure it cannot be weaponised against innocent people.

Secondary Stakeholder: Emergency Contacts

Their core problem is simple: they do not know when something is wrong or where the person is. The location sharing (SR-F4) and notification system (SR-F10) fix this by getting them an exact location and a clear alert within seconds of an emergency being triggered.

Tertiary Stakeholder: Innocent Bystanders

This is a group most safety apps completely ignore. Anyone near a user who triggers the app is potentially affected if the system can be misused. The entire situation classification pipeline including SR-F8, SR-NF3 and UC-06 exists specifically to protect this group. By making sure recordings never leave the app unless an AI review confirms a genuine threat, we remove the ability to use the app as a weapon against someone who did nothing wrong.

Quaternary Stakeholder: Academic Evaluators and Future Developers

The three ML components, voice trigger detection, risk zone classification, and situation classification, are all genuine AI use cases that are realistic to build within a student project. All three run on the device, require no cloud processing, and do not collect data from anyone who has not consented. The ethical design is not an add on. It is part of the core functionality.

Task 4: Gradio/Streamlit Demo

Public URL for our Gradio demo: <https://91893c6a66766fa86d.gradio.live/>

URL of our Google Colab notebook containing the code of the demo:

<https://colab.research.google.com/drive/1Mkgj2cCPytqO1ESJcEZRu8-6g4fB3uI7?usp=sharing>